

Contents

Cryptography & Applications — Laboratory Series (Labs 1–6).....	1
Course Laboratory Objectives	1
Lab 1 — Symmetric Encryption with Crypto++	10
Lab 2 — AES-128 Implementation in Pure C++ (CTR Mode, FIPS-197 Compliant)	18
Lab 3 — RSA-OAEP & Hybrid Encryption (Crypto++ / OpenSSL)	25
Lab 4 — Hashing, PKI, and Practical Attacks	31
Lab 5 — Classical Digital Signatures (ECDSA, RSA-PSS)	38
Lab 6— Post-Quantum Signatures & Certificates (ML-DSA, ML-KEM)	43

Cryptography & Applications — Laboratory Series (Labs 1–6)

Course Level: Upper Undergraduate / Early Graduate

Platforms: Windows 10/11 and Linux (Ubuntu LTS recommended)

Languages: Modern C++ (C++17 or later)

Libraries: Crypto++ and/or OpenSSL (as specified per lab)

Optional GUI: Python (PySide6/Qt) or C# (WPF/WinUI)

Build System: CMake (required)

Submission Format: Single ZIP/RAR archive

Course Laboratory Objectives

Across Labs 1–6, students will:

1. Implement and analyze modern cryptographic primitives
2. Apply secure engineering practices in CLI-based tools
3. Validate correctness using official NIST test vectors
4. Detect and prevent common cryptographic misuse
5. Perform statistically sound performance benchmarking
6. Produce professional-quality technical reports

Submission Structure (All Labs). MSV.All6labs.rar

Single compressed archive must contain:

- Source code for all labs

- CMake build system
- README.md (top-level)
- Report (single PDF/DOCX covering all labs)
- Binaries for Windows and Linux
- Unit tests
- Build scripts
- Any required resource files

1. Common Engineering Requirements

These apply to **all labs**.

A. Environment & Tooling

Build Requirements

- Must use **CMake**
- Must support out-of-source builds:
 - mkdir build && cd build
 - cmake ..
 - cmake --build .
- Must compile successfully on:
 - Ubuntu LTS
 - Windows (MSVC and MinGW64)

README.md Must Include

- Dependency list (with versions)
- Installation instructions (Windows & Linux)
- Build commands for both OSes
- Example CLI usage
- Known limitations

CLI Standard (Uniform Across Labs)

All tools must follow a consistent structure:

mytool <command>

[--in INFILE | --text "..."]

[--out OUTFILE]

[--key KEYFILE | --key-hex HEX]

[--iv IV-hex]

[--nonce NONCE-hex]

[--mode MODE]

[--aad]

[--encode hex|base64|raw]

[--threads N]

[--kat path/to/vectors.json]

[--verbose]

Programs must:

- Accept UTF-8 input
- Produce UTF-8 output
- Fail closed on malformed input

B. Input/Output & Encoding

Input

- --in file (binary-safe)
- --text "..." (UTF-8)

Output

- --out file (binary-safe)
- On-screen output defaults to:
 - Hex for ciphertext, digest, signatures
- File output defaults to:
 - Raw binary (unless overridden)

Encoding Options

--encode hex
--encode base64
--encode raw

2. Randomness & Key Management

Secure Randomness

Must use:

- Crypto++ AutoSeededRandomPool
- OpenSSL RAND_bytes

Never use:

- rand()
- std::random for cryptographic purposes

IV / Nonce Rules

- Must enforce proper IV length
- Must prevent nonce reuse when prohibited
- Must auto-generate securely if missing (where appropriate)
- Must document storage format (header/sidecar)

Minimum Key Sizes

Algorithm Requirement

AES 128/192/256 bits (prefer 256 in reports)

RSA ≥ 3072 bits

ECDSA $\geq$ P-256 (secp256r1)

3. Verification & Testing

Known Answer Tests (KATs)

Each lab must include:

- NIST test vectors where applicable
- A --kat runner
- JSON vector format

Example:

```
mytool --kat vectors.json
```

Must output:

- PASS/FAIL per test
- Summary statistics

Negative Testing

Students must include:

- Wrong key tests
- Tampered ciphertext tests
- Tampered authentication tag tests (AEAD)
- Malformed input tests
- Corrupted PEM/key file tests (where applicable)

Programs must fail securely.

Unit Testing

- Must include Catch2 or GoogleTest
- ctest must pass on Windows and Linux
- CI integration recommended (optional)

4. Performance Methodology (Standardized)

For each cryptographic algorithm:

1. Warm-up: 1–2 seconds
2. Execute a block of ~1,000 operations
3. Repeat 30–100 times
4. Collect statistics

Required Statistical Reporting

Report must include:

- Mean
- Median
- Standard deviation
- 95% confidence interval

Comparative Analysis

Where applicable, compare:

- Windows vs Linux
- Algorithm variants
- Mode overhead
- Hardware acceleration impact

Include:

- Tables
- Plots
- Interpretation (not just raw numbers)

5. Security Engineering Standards

Each lab report must discuss:

- Threat model
- Misuse scenarios

- Known attacks relevant to the algorithm
- Secure defaults
- Fail-closed behavior
- Limitations of the implementation

Students must demonstrate understanding beyond implementation.

6. Deliverables Checklist

Final archive must include:

- Source code
- CMake files
- README
- Unit tests
- Windows binary
- Linux binary
- Report (PDF/DOCX)
- Self-grade checklist
- Academic integrity statement

7. Report Requirements (Single File for All Labs)

The report must contain:

- Objectives per lab
- Implementation details
- Test results
- Performance methodology
- Statistical tables & plots
- Security analysis
- Cross-platform comparison

- Lessons learned

Minimum expected length:

- 60 pages total (typical)

8. Optional GUI Bonus

Students may:

- Export crypto core as:
 - .dll
 - .so
 - .a
 - .lib
- Build GUI in:
 - Python (PySide6/Qt)
 - C# (WPF/WinUI)

GUI must:

- Call the same compiled library
- Not duplicate cryptographic logic

Bonus awarded per lab specification.

9. Academic Integrity & Ethics

Students must include:

- Statement of originality
- Disclosure of AI/tool assistance (if used)
- Proper citation of external references
- No copying of external implementations

Violation results in academic penalty.

Expected Program-Level Learning Impact

Upon completing Labs 1–6, students should:

- Understand symmetric and asymmetric cryptography deeply
- Implement cryptographic systems safely
- Recognize real-world misuse patterns
- Perform statistically sound benchmarking
- Produce professional-grade technical documentation
- Bridge theory and engineering practice

Lab 1 — Symmetric Encryption with Crypto++

⚠ Constraint

- **Crypto++ library must be used for cryptographic primitives**
- **No use of OpenSSL, libsodium, or other crypto libraries**
- **C++ STL permitted**
- **Must compile on Windows and Linux**
- **CMake build system required**

Learning Outcomes

After completing this lab, students will be able to:

- 1. Use Crypto++ safely across multiple block and stream cipher modes**
- 2. Correctly implement IV/nonce handling and misuse prevention**
- 3. Design robust file I/O and encoding pipelines**
- 4. Validate implementations using official NIST test vectors**
- 5. Benchmark symmetric encryption performance**
- 6. Detect and handle common cryptographic misuse (e.g., nonce reuse)**

1. Algorithm & Implementation Scope

A. AES Modes (Runtime Selectable)

The tool must support the following modes:

- **ECB**
- **CBC**
- **OFB**
- **CFB**
- **CTR**

- XTS
- CCM (AEAD)
- GCM (AEAD)

Mode selection must occur via CLI flag:

`--mode ecb|cbc|ofb|cfb|ctr|xts|ccm|gcm`

B. AEAD Handling

When using AEAD modes (CCM, GCM):

- `--aad` flag must enable authenticated behavior
- Support:
 - `--aad FILE`
 - `--aad-text STRING`
- Must verify authentication tag during decryption
- If tag verification fails → fail closed

C. Misuse Prevention Requirements

Students must implement explicit misuse checks:

ECB Restrictions

- Print WARNING when ECB is selected
- Block files larger than 16 KiB by default
- Allow override only with:
 - `--allow-ecb`

IV / Nonce Handling

For modes requiring IV/nonce:

- Enforce correct length per mode
- If IV/nonce is omitted:

- Securely generate using **Crypto++ AutoSeededRandomPool**
 - Persist IV to output header or sidecar JSON
- Reject invalid IV length

Nonce Reuse Protection

For CTR, CCM, GCM:

- If output header indicates same key + nonce previously used → reject operation
- Must document detection logic

2. CLI & I/O Requirements

Encryption

aestool encrypt --mode gcm --key key.bin --in msg.txt --out ct.bin

Decryption

aestool decrypt --mode gcm --key key.bin --in ct.bin --out msg.txt

Key Handling

Support:

--key-hex HEXSTRING

--key KEYFILE

Key file may be:

- Raw binary
- Hex-encoded with header

IV / Nonce Handling

--iv IVFILE

--nonce NONCEFILE

If omitted (except ECB/XTS):

- **Must auto-generate securely**
- **Must persist to header or sidecar file**

Input & Encoding

- **Input from file or --text**
- **Output:**
 - **Raw ciphertext written to file**
 - **Hex or Base64 displayed to screen**
- **UTF-8 safe**

Optional Sidecar JSON Header

Example:

```
{  
  "alg": "AES-256-GCM",  
  "iv": "...hex...",  
  "aad": "...hex...",  
  "tag": "...hex..."  
}
```

Header must include:

- **Algorithm**
- **Mode**
- **IV/nonce**
- **AAD (if any)**
- **Authentication tag (AEAD modes)**

3. Correctness & Validation

A. Known Answer Tests (KATs)

Students must validate against:

- **NIST SP 800-38A vectors:**
 - **CBC**
 - **CFB**
 - **OFB**
 - **CTR**
- **NIST GCM vectors**
- **NIST CCM vectors**

B. KAT Runner

Implement:

--kat vectors.json

Must:

- **Parse JSON vector file**
- **Run test cases**
- **Output PASS/FAIL per case**
- **Provide summary**

C. Negative Tests

Students must demonstrate:

- **Wrong key → incorrect plaintext**
- **Wrong IV → incorrect plaintext**
- **Tampered ciphertext (non-AEAD) → corrupted output**
- **Tampered ciphertext (AEAD) → authentication failure**
- **Invalid tag → decryption refusal**
- **Invalid IV length → rejection**

All malformed inputs must fail closed.

4. Performance Evaluation

Students must benchmark:

Payload sizes:

- **1 KB**
- **4 KB**
- **16 KB**
- **256 KB**
- **1 MB**
- **8 MB**

Metrics:

- **Throughput (MB/s)**
- **Latency per operation**

Required Comparative Study

Compare:

- **Windows vs Linux**
- **Stream vs block modes**
- **AEAD vs non-AEAD**
- **Tag overhead (GCM/CCM)**

Include:

- **Tables**
- **Plots**
- **Analysis of OS scheduling and system call overhead**

5. Security Engineering Discussion (Mandatory Report Section)

Students must analyze:

Mode-Level Security

- **Why ECB is insecure**
- **CBC padding oracle risks**
- **CTR nonce reuse catastrophe**
- **AEAD guarantees vs non-AEAD modes**
- **Why GCM requires unique nonces**
- **XTS limitations (no integrity)**

Implementation-Level Security

- **Proper randomness generation**
- **Safe IV storage**
- **Authentication tag verification**
- **Failure handling (fail closed)**
- **Key storage risks**

Cross-Platform Considerations

- **RNG differences between OSes**
- **File system behavior**
- **Performance variation causes**

6. Optional Extension (+5 Bonus)

Students may:

- **Export encryption core as:**
 - **.dll (Windows)**
 - **.so (Linux)**
 - **.a / .lib**

- **Build minimal GUI in:**
 - **Python**
 - **C#**
 - **GUI must call compiled library**
-

Rubric (100 pts)

Component	Points
Correctness & KATs	25
Security hygiene	15
UX & I/O design	10
Performance methodology	20
Cross-platform build	10
Report quality	20
Bonus: Library export + GUI +5	

Expected Learning Impact

After completing this lab, students should understand:

- **Practical use of symmetric encryption libraries**
- **Differences between block, stream, and AEAD modes**
- **Proper IV/nonce lifecycle management**
- **Real-world cryptographic misuse risks**
- **Performance trade-offs across OS platforms**
- **Importance of authentication in modern encryption**

Lab 2 — AES-128 Implementation in Pure C++ (CTR Mode, FIPS-197 Compliant)

⚠ Constraint

- **No external cryptographic libraries allowed**
- C++ STL is permitted
- All cryptographic primitives must be implemented manually
- No use of OpenSSL, Crypto++, libsodium, etc.
- AES-NI may only be used in bonus section

Learning Outcomes

After completing this lab, students will be able to:

1. Implement AES-128 round transformations according to FIPS-197
2. Implement CTR (Counter) mode correctly per NIST SP 800-38A
3. Validate correctness using official NIST Known Answer Tests
4. Reason about nonce misuse and stream-cipher risks
5. Benchmark software AES performance
6. Analyze side-channel leakage in table-based implementations
7. Understand why CTR requires authentication

1. Algorithm & Implementation Scope

A. AES Block Cipher (FIPS-197, May 9, 2023 Update)

Required: AES-128

Students must implement:

- AddRoundKey

- SubBytes (S-box lookup)
- ShiftRows
- MixColumns
- KeyExpansion

Parameters:

- Block size: 128 bits
- Key size: 128 bits
- Number of rounds: 10

Implementation must strictly follow FIPS-197 specification.

Bonus Extension (+10)

- AES-192
- AES-256
- Full key schedule implementation required
- KAT validation mandatory

B. CTR Mode (NIST SP 800-38A Compliant)

CTR turns AES into a stream cipher.

Correct CTR Definition

For block index (i):

[
 $S_i = \text{AES}(K, \text{Nonce} \parallel \text{Counter}_i)$
]

[
 $C_i = P_i \oplus S_i$
]

[
 $P_i = C_i \oplus S_i$
]

Requirements

- 128-bit IV (Nonce + Counter)
- Counter must increment correctly
- Must clearly document:
 - Counter endianness
 - Counter overflow behavior
- Must support arbitrary-length plaintext
- Partial final blocks must be handled correctly
- No padding required

Critical Security Rule

IV (nonce) must be **unique per key**.

Reuse of IV under the same key must be discussed as catastrophic.

1. CLI & Input/Output Requirements

Encryption

```
aestool encrypt --mode ctr --key key.bin --iv iv.bin --in msg.bin --out ct.bin
```

Decryption

```
aestool decrypt --mode ctr --key key.bin --iv iv.bin --in ct.bin --out msg.bin
```

Supported Formats

- Input/output: raw binary
- Optional: hex encoding for debugging
- Key: exactly 16 bytes
- IV: exactly 16 bytes

Input Validation (Fail Closed)

Students must validate:

- Invalid key length
- Invalid IV length
- Counter overflow detection (if applicable)
- File I/O errors

CTR does NOT require:

- Padding
- Block alignment
- Padding validation

Any malformed input must terminate execution securely.

3. Correctness & Validation

A. Known Answer Tests (KATs)

Students must verify against:

- Official FIPS-197 AES test vectors
- NIST SP 800-38A CTR mode test vectors

B. Cross-Validation

Implementation must produce identical ciphertext to:

- Crypto++ AES-128-CTR
OR
- OpenSSL AES-128-CTR

(using identical key and IV)

C. Negative Tests

Students must demonstrate:

- Wrong key → incorrect plaintext
- Wrong IV → incorrect plaintext
- Tampered ciphertext → corrupted plaintext

Students must explain why CTR cannot detect tampering.

4. Performance Evaluation

Students must benchmark:

- AES-128 encryption throughput
- AES-128 decryption throughput

File sizes:

- 1 MiB
- 100 MiB
- 1 GiB (if feasible)

Required Analysis

Students must analyze:

- Table-based S-box vs computed S-box
- Memory footprint
- Cache behavior
- Branch prediction impact
- Comparison with Crypto++ performance

Include:

- Throughput table (MB/s)
- Throughput plots

5. Security Engineering Discussion (Mandatory Report Section)

Students must analyze:

AES Core Risks

- Timing leakage from table-based S-box
- Cache-based attacks (T-table leakage)
- Importance of constant-time coding

CTR Mode Risks

- Keystream reuse attack
- Two-time pad vulnerability
- Why IV reuse is catastrophic
- Why CTR provides confidentiality only
- Why CTR requires MAC for authenticity
- Why CTR is not AEAD

Hardware Acceleration

- Why AES-NI mitigates many software leakage risks
- Performance vs security trade-offs

6 Advanced Topics (+20 Bonus)

A. AES-192 / AES-256 (+10)

- Full key expansion
- KAT validation required

B. AES-XTS Implementation (+5)

Implement AES-XTS:

- Two-key construction
- Tweak multiplication in $GF(2^{128})$
- Explain disk encryption use case

C. AES-NI or Bit-Sliced Optimization (+5)

- Runtime CPU feature detection
- Compare performance vs pure software

- Explain constant-time improvements

Rubric (100 pts + Bonus)

Component	Points
FIPS-197 correct AES-128 core	30
CTR mode correctness	15
KATs & cross-validation	15
Engineering quality & documentation	10
Performance study & analysis	10
Security discussion	20
Bonus: AES-192/256	+10
Bonus: XTS	+5
Bonus: AES-NI/bit-sliced	+5

Expected Learning Impact

After completing this lab, students should understand:

- Internal structure of AES rounds
- How stream cipher modes operate
- Why nonce reuse destroys confidentiality
- Why CTR must be combined with MAC
- Why constant-time implementations matter
- How hardware acceleration changes security posture
- for students

Lab 3 — RSA-OAEP & Hybrid Encryption (Crypto++ / OpenSSL)

Learning Outcomes

By the end of this lab, students will be able to:

1. Generate and securely store RSA key pairs
2. Implement RSA-OAEP encryption/decryption (SHA-256)
3. Handle plaintext size limits and implement hybrid encryption
4. Detect and properly handle cryptographic failures
5. Benchmark RSA performance across key sizes
6. Understand why hybrid encryption is required in practice

1. Algorithms & Parameters

A. RSA-OAEP (Asymmetric Encryption)

Parameter	Required Setting
-----------	------------------

Modulus size	≥ 3072 bits
--------------	------------------

Hash function	SHA-256
---------------	---------

Padding	OAEP
---------	------

MGF	MGF1 (SHA-256)
-----	----------------

Required

- Implement RSA-3072 with OAEP(SHA-256)
- Add RSA-4096 for performance comparison
- Support optional OAEP label

Required Discussion (Report)

- Why OAEP is secure (IND-CCA2)
- Why PKCS#1 v1.5 encryption is insecure
- Plaintext size limit:

```
[  
mLen \le k - 2hLen - 2  
]
```

- Why RSA cannot encrypt large files directly

B. Hybrid Encryption (Envelope Encryption)

For plaintext exceeding RSA-OAEP limit:

1. Generate random AES key (256-bit)
2. Encrypt data using AES-GCM
3. Encrypt AES key using RSA-OAEP
4. Output structured envelope

This mirrors real-world TLS and PGP behavior.

2. CLI & Formats

Key Generation

```
rsatool keygen --bits 3072 --pub pub.pem --priv priv.pem
```

Must also save:

- PEM format
- DER format
- Metadata JSON:

```
{  
  "creation_time": "...",  
  "modulus_bits": 3072,  
  "hash": "SHA-256"  
}
```

Encryption

```
rsatool encrypt --in msg.bin --pub pub.pem --out ct.bin --label optional
```

If message exceeds RSA limit:

- Automatically switch to hybrid mode

- **Output envelope**

Decryption

rsatool decrypt --in ct.bin --priv priv.pem --out msg.bin

Supported Formats

- **Ciphertext: raw / hex / base64**
- **Envelope header: JSON**
- **Keys: PEM / DER**

Students must validate:

- **Malformed ciphertext**
- **Incorrect label**
- **Unsupported key sizes**
- **Incorrect encoding**

3. Hybrid Envelope Format

Example JSON header:

```
{  
  "mode": "RSA-OAEP-AES-GCM",  
  "rsa_modulus": 3072,  
  "hash": "SHA-256",  
  "wrapped_key": "<base64>",  
  "iv": "<base64>",  
  "tag": "<base64>"  
}
```

Payload:

- **AES-GCM encrypted data**

Required Security Properties

- **AES-256-GCM**
- **96-bit IV**

- **Authenticated decryption**
- **Reject tampered ciphertext**
- **Constant-time tag verification (library-backed acceptable)**

4. Correctness & Negative Tests

Students must demonstrate:

- **Altered RSA ciphertext → decryption fails**
- **Altered AES-GCM ciphertext → tag failure**
- **Wrong private key → failure**
- **Wrong OAEP label → failure**
- **Tampered envelope header → failure**

Include:

- **Automated unit tests**
- **Clear, non-leaky error messages**
- **Fail closed on parsing errors**

5. Performance Evaluation

Students must benchmark:

RSA

- **Key generation time**
- **Encryption time**
- **Decryption time**
- **3072 vs 4096 comparison**

Hybrid Mode

- **AES-GCM encryption throughput**
- **Hybrid encryption time for:**
 - **1 KiB**
 - **1 MiB**
 - **100 MiB**

Required Analysis

- **RSA decryption vs encryption cost**
- **RSA key size vs performance tradeoff**
- **Hybrid efficiency advantage**
- **Why symmetric encryption dominates throughput**
- **Memory and CPU considerations**

Include latency and throughput plots.

6. Advanced Topics (+15 Bonus)

A. Manual OAEP Padding (+5)

Implement OAEP encoding/decoding manually:

- **MGF1**
- **XOR masking**
- **Padding validation**

Explain why decoding must be constant-time.

B. AES + RSA Hybrid Security Analysis (+10)

Provide deeper hybrid study:

- **Compare envelope vs direct RSA**
- **Explain how TLS handshake uses hybrid design**
- **Analyze forward secrecy limitations of pure RSA**
- **Propose upgrade path to ECDHE or ML-KEM (preview of Lab 6)**

Optional extension:

- **Add AES-CTR + HMAC comparison vs AES-GCM**
- **Measure authentication failure timing variance**

Rubric (100 pts + Bonus)

Component	Points
Correct RSA-OAEP (SHA-256)	20
Correct hybrid envelope (AES-GCM + RSA)	20
Key management & formats	15
Negative tests & secure error handling	15
Performance study & analysis	15
Cross-platform build & documentation	5
Report quality	10
Bonus: manual OAEP	+5
Bonus: hybrid security analysis	+10

Expected Learning Impact

After completing this lab, students should understand:

- **Why RSA must use OAEP**
- **Why hybrid encryption is mandatory for large data**
- **Why symmetric crypto dominates performance**
- **Real-world envelope encryption design**
- **RSA limitations compared to modern KEMs (preview of ML-KEM in Lab 6)**

Lab 4 — Hashing, PKI, and Practical Attacks

⚠ Ethics Notice

This lab is for defensive education only.

Use offline test files you own, in an isolated environment.

Do NOT target real systems or live services.

Learning Outcomes

By the end of this lab, students will be able to:

1. Implement and benchmark modern hash functions
2. Parse and validate X.509 certificates
3. Demonstrate MD5 collision weaknesses safely
4. Demonstrate length-extension attacks on naïve MAC constructions
5. Explain why legacy hash functions are insecure
6. Propose cryptographic mitigations (HMAC, SHA-2/3, policy controls)

1. Hashing Suite

Students must implement a CLI hashing tool supporting:

A. SHA-2 Family

Algorithm Output Size

SHA-224 224 bits

SHA-256 256 bits

SHA-384 384 bits

SHA-512 512 bits

B. SHA-3 Family

Algorithm Output Size

SHA3-224 224 bits

SHA3-256 256 bits

Algorithm Output Size

SHA3-384 384 bits

SHA3-512 512 bits

C. SHAKE (Extendable-Output Functions)

Algorithm Output

SHAKE128 Variable

SHAKE256 Variable

Must support:

hashtool --algo shake256 --outlen 64 --in file.bin

Required Implementation Features

- Known Answer Tests (KATs)
- Streamed I/O (multi-GB files supported)
- Output:
 - hex (console)
 - raw binary (file)

Required Discussion

- Merkle–Damgård vs sponge construction
- Collision resistance vs preimage resistance
- Why SHA-3 differs structurally from SHA-2
- XOF use cases (e.g., domain separation)

2. CLI & Formats

Basic Hashing

hashtool --algo sha256 --in file.bin

Streaming Mode

hashtool --algo sha512 --in large.iso --stream

SHAKE Output Length

hashtool --algo shake128 --outlen 128 --in file.bin

Students must:

- Handle malformed inputs
- Validate algorithm identifiers
- Fail closed on unsupported parameters

3. PKI & Certificate Analysis

Students must parse an **X.509 certificate** using OpenSSL or Crypto++.

Required Extraction

From the certificate:

- Subject
- Issuer
- Subject Public Key Info (algorithm & parameters)
- Signature algorithm
- Validity period
- Key usage
- Subject Alternative Names (SANs)

Signature Verification

Students must:

- Verify certificate signature using issuer public key (if provided)
- If issuer key unavailable:
 - Verify TBS structure integrity
 - Validate algorithm consistency
 - Fail closed

TLS Deployment Task

Configure HTTPS on:

- Apache or Nginx

- TLS 1.2 or TLS 1.3
- Using a trusted root (e.g., ZeroSSL)
- With ECDSA certificate (preferred)

Deliver:

- Screenshot or logs
- Config snippet
- Short explanation of trust chain

Required Discussion

- Structure of X.509 (TBS vs signature)
- Chain of trust
- Why SHA-1 and MD5 are banned
- Certificate transparency & CA/B requirements

4. Controlled MD5 Collision Demonstration

Students must demonstrate an MD5 collision using **hashclash**.

Choose ONE:

Option A

Two benign PNG files with identical MD5 digest

Option B

Two short C++ programs with identical MD5 digest

Deliver:

- Both input files
- Matching MD5 digest
- One-page explanation:
 - Why MD5 is broken
 - Collision vs preimage difference
 - Historical incidents

No real-world targets allowed.

5. Length-Extension Attack on Naïve MAC

Target construction:

[
MAC = $H(k \parallel m)$
]

Using **hashpump** (or own implementation, bonus +5):

Demonstrate:

Given:

- $H(k \parallel m)$
- Length of k

Compute:

- $H(k \parallel m \parallel \text{pad} \parallel m')$

Without knowing k .

Deliverables

- Original message
- Forged extended message
- Matching forged MAC
- Diagram of padding
- Mitigation explanation

Required Discussion

- Why Merkle–Damgård enables length extension
- Why HMAC prevents it
- Prefix-free constructions

6. Performance Evaluation

Students must benchmark:

- SHA-256
- SHA-512
- SHA3-256

- SHA3-512

File Sizes

- 1 MiB
- 100 MiB
- 1 GiB (if feasible)

Required Analysis

- Throughput (MB/s)
- Streaming vs memory-mapped I/O
- CPU utilization
- SHA-2 vs SHA-3 performance differences
- Cache and memory effects

Include performance plots.

Test on both OSes (Windows + Linux if possible).

7. Advanced Topics (+15 Bonus)

A. Custom Length-Extension Implementation (+5)

Implement length-extension manually for SHA-256:

- Reconstruct internal state
- Apply padding
- Continue compression

Explain internal state manipulation.

B. Chosen-Prefix Certificate Demo (+10)

In sandbox only:

- Generate MD5/RSA self-signed cert
- Use chosen-prefix collision
- Craft second certificate with same MD5 signature

Deliver:

- Explanation of why MD5-signed certificates are catastrophic

- Policy mitigations:
 - Ban MD5/SHA-1
 - Enforce CA/B Baseline Requirements
 - Algorithm agility

No operational exploit required.

Rubric (100 pts + Bonus)

Component	Points
Hash suite correctness & KATs	25
Streaming & large-file performance	10
X.509 parsing & signature verification	25
MD5 collision demo & explanation	20
Length-extension demo & mitigation analysis	20
Bonus: custom length-extension	+5
Bonus: chosen-prefix certificate demo	+10

Expected Learning Impact

After completing this lab, students should understand:

- Structural differences between SHA-2 and SHA-3
- Why MD5 and SHA-1 are cryptographically broken
- How naive MAC constructions fail
- Why HMAC is secure
- How certificate validation works internally
- How cryptographic failures propagate to system-level vulnerabilities

Lab 5 — Classical Digital Signatures (ECDSA, RSA-PSS)

Learning Outcomes

By the end of this lab, students will be able to:

1. Generate public/private key pairs for ECDSA and RSA-PSS
2. Produce and verify detached digital signatures
3. Select secure parameters and hash functions
4. Detect implementation failures and signature misuse
5. Benchmark signing and verification performance
6. Compare elliptic-curve vs RSA-based signature systems
7. Libraries: Cryptopp + OpenSSL

1. Algorithms & Parameters

A. ECDSA (Elliptic Curve Digital Signature Algorithm)

Based on NIST prime curves.

Security Level	Curve	Approx. Classical Strength
Level 1	secp256r1 (P-256)	~128-bit
Level 3	secp384r1 (P-384)	~192-bit

Required

- Implement **ECDSA-P256**
- Add **ECDSA-P384** (+5 pts)

Required Discussion (Report)

- Deterministic nonces (RFC 6979) vs random nonces
- Catastrophic nonce reuse vulnerability
- Signature size vs RSA-PSS
- Verification cost vs signing cost

Students must use **deterministic ECDSA (RFC 6979)** unless justified otherwise.

B. RSA-PSS (Probabilistic Signature Scheme)

Parameter Required Setting

Modulus size 3072 bits

Hash function SHA-256

Salt length hashLen (32 bytes)

Required

- Implement **RSA-PSS-3072 with SHA-256**
- Use randomized salt per signature

Required Discussion

- Why RSA-PSS is preferred over PKCS#1 v1.5
- Role of salt in probabilistic signatures
- Signature size vs ECDSA
- Public exponent choice (e.g., 65537)

2. CLI & Formats

CLI structure mirrors Lab 6 for consistency.

Key Generation

```
sigtool keygen --algo ecdsa-p256 --pub pub.pem --priv priv.pem
```

```
sigtool keygen --algo rsa-pss-3072 --pub pub.pem --priv priv.pem
```

Signing

```
sigtool sign --algo ecdsa-p256 --in msg.bin --out sig.bin --hash sha256
```

Verification

```
sigtool verify --algo ecdsa-p256 --in msg.bin --sig sig.bin --pub pub.pem
```

Supported Formats

- Keys: PEM / DER
- Signatures: raw / DER / base64 (--encode)
- Hash: SHA-256 required (SHA-384 optional for P-384)

Students must validate:

- Malformed keys
- Incorrect algorithm identifiers
- Unsupported encodings
- Hash mismatch errors

3. Correctness & Negative Tests

Students must demonstrate:

- Modified message → verification fails
- Modified signature → fails
- Modified public key → fails
- Wrong algorithm identifier → fails
- Wrong hash function → fails

Include:

- Automated unit tests
- Batch verification (verify N signatures)
- Clear error codes and UX messages

Required Discussion

- Why signature verification must be constant-time
- How improper error handling may leak information

4. Performance Evaluation

Students must benchmark:

ECDSA

- Key generation latency
- Sign latency
- Verify latency
- Throughput (ops/sec)

RSA-PSS

- Key generation latency

- Sign latency
- Verify latency
- Throughput (ops/sec)

Message Sizes

- 1 KiB
- 16 KiB
- 1 MiB
- 8 MiB

Required Analysis

- Hash cost dominance for large messages
- ECDSA faster verification vs RSA-PSS characteristics
- Signature size comparison
- Memory usage considerations

Students must compare results against Lab 6 (PQC) in discussion section.

Encourage graphical plots (latency vs message size).

5. Advanced Topics (+15 Bonus)

Students may implement:

A. Formula-Level Implementation

From primitives:

- Elliptic curve point arithmetic
- Modular inversion
- RSA modular exponentiation
- PSS padding generation

Explain constant-time considerations and potential side-channel risks.

B. Security Engineering Considerations

Explain:

- Timing attack surface

- Fault injection risks
- Side-channel risks in modular exponentiation
- Deterministic nonce protection

Partial validation:

- Basic timing variance measurement
- Discussion of constant-time libraries

(No exploit demonstration required.)

Rubric (100 pts + Bonus)

Component	Points
Correct ECDSA sign/verify	20
Correct RSA-PSS sign/verify	20
Robust key handling & formats	15
Negative tests & UX	10
Performance study & analysis	20
Cross-platform build & documentation	5
Report quality	10
Bonus (advanced implementation)	+15

Expected Learning Impact

After completing this lab, students should understand:

- Differences between elliptic-curve and RSA signatures
- Importance of deterministic nonces in ECDSA
- Why RSA-PSS improves security over PKCS#1 v1.5
- Practical trade-offs in signature size and speed
- Engineering challenges in secure cryptographic implementation

Lab 6— Post-Quantum Signatures & Certificates (ML-DSA, ML-KEM)

By the end of this lab, students will:

1. Generate PQC key pairs (ML-DSA, ML-KEM)
2. Sign and verify detached signatures using ML-DSA
3. Perform encapsulation/decapsulation using ML-KEM
4. Understand PQC parameter selection and security levels
5. Benchmark PQ signature and KEM performance
6. Build a simple PQ certificate structure (ML-DSA-signed public keys)

1. Algorithms & Parameters

A. ML-DSA (Post-Quantum Signatures)

Security Level Parameter Set Equivalent Classical

Level 2	ML-DSA-44	~128-bit
Level 3	ML-DSA-65	~192-bit
Level 5	ML-DSA-87	~256-bit

Students must:

- Implement ML-DSA-44 (required)
- Add ML-DSA-65 (+5 pts)
- Discuss signature size vs security trade-offs

Key discussion:

- Large signature size (compared to ECDSA)
- Deterministic signing (no RFC6979 issue like ECDSA)
- Rejection sampling and failure probability

B. ML-KEM (Post-Quantum KEM)

Security Level Parameter Set Equivalent Classical

Level 2	ML-KEM-512
Level 3	ML-KEM-768
Level 5	ML-KEM-1024

Students must:

- Implement ML-KEM-512
- Perform encapsulation → shared secret
- Perform decapsulation → verify shared secret match

Discuss:

- IND-CCA security
- Fujisaki–Okamoto transform
- Ciphertext size vs RSA key exchange

2. CLI & Formats

Key Generation

```
pqtool keygen --algo mldsa-44 --pub pub.pem --priv priv.pem
```

```
pqtool keygen --algo mlkem-512 --pub pub.pem --priv priv.pem
```

Signing

```
pqtool sign --algo mldsa-44 --in msg.bin --out sig.bin
```

Verification

```
pqtool verify --algo mldsa-44 --in msg.bin --sig sig.bin --pub pub.pem
```

KEM Encapsulation

```
pqtool encaps --algo mlkem-512 --pub pub.pem --ct ct.bin --ss ss.bin
```

KEM Decapsulation

```
pqtool decaps --algo mlkem-512 --priv priv.pem --ct ct.bin --ss ss.bin
```

Supported Formats

- Keys: PEM / DER
- Signatures: raw / base64
- Ciphertexts: binary
- Optional: JSON structured certificate format

3. Post-Quantum Certificate Mini-Project

Students must implement a minimal certificate structure:

```
{
  "subject": "your Subject",
  "public_key": "<ML-DSA public key>",
  "issuer": "PQ-CA",
  "signature": "<ML-DSA signature by CA>"
}
```

Tasks:

1. Generate CA key pair (ML-DSA)
2. Sign subject public key
3. Verify certificate signature
4. Detect tampering

Discuss:

- Why ML-KEM is NOT used for signing
- Hybrid certificates (ECDSA + ML-DSA)
- Migration path to PQ TLS

4. Correctness & Negative Tests

Students must demonstrate:

- Modified message → verification fails
- Modified signature → fails
- Modified public key → fails
- Modified ciphertext (KEM) → decapsulation fails

- Wrong private key → failure

Include:

- Automated unit tests
- Batch verification (verify N signatures)
- Batch decapsulation timing

5. Performance Evaluation

Measure:

ML-DSA

- Keygen latency
- Sign latency
- Verify latency
- Throughput (ops/sec)

ML-KEM

- Keygen latency
- Encapsulation latency
- Decapsulation latency
- Shared secret generation throughput

Test message sizes:

- 1 KiB
- 16 KiB
- 1 MiB
- 8 MiB

Discuss:

- Hash cost dominance
- Large signature overhead
- Memory usage
- Compare to ECDSA/RSA-PSS (theoretical comparison acceptable)

6. Advanced Topics (+15 Bonus)

Students may implement:

A. Formula-Level ML-DSA or ML-KEM (from primitives)

- NTT-based polynomial arithmetic
- Modular reduction
- Rejection sampling
- Constant-time coding considerations

B. Side-Channel Awareness

Explain:

- Timing attack surface
- Lattice leakage risks
- Importance of constant-time NTT
- Failure handling in decapsulation

Partial validation:

- Basic timing variance measurement
- Input-dependent timing discussion

Rubric (100 pts + Bonus)

Component	Points
Correct ML-DSA sign/verify	25
Correct ML-KEM encaps/decaps	20
Robust key handling & formats	15
Certificate implementation	10
Negative tests & UX	10
Performance study & analysis	15

Component	Points
Cross-platform build & documentation	5
Report quality	10
Bonus (advanced implementation)	+15

Expected Learning Impact

Students will understand:

- Why PQ signatures are large
- How lattice cryptography works at high level
- How ML-KEM replaces RSA key exchange
- Migration strategy toward PQ TLS
- Practical engineering trade-offs

Reporting Template (use & adapt)

- 1. Objective & outcomes** — what you built and why.
 - 2. Design & security** — parameters, mode choices, misuse checks.
 - 3. Test methodology** — vectors used (cite), negative tests.
 - 4. Performance** — hardware table (CPU, RAM, OS, compiler flags), protocol, plots (throughput MB/s; latency ms/op), 95% CIs.
 - 5. Discussion** — correctness issues found and fixed; cross-OS comparisons; security trade-offs.
 - 6. Conclusion** — key takeaways & future work.
- Appendix** — CLI usage examples; build logs; KAT coverage; unit test summary.

Performance Protocol (copy into each lab)

- **Hardware/OS table:** CPU model, cores/threads, RAM, storage type, OS version, compiler & flags.
- **Warm-up:** 1–2 s to stabilize caches and JIT/allocators the run 1000 rounds per block.
- **Runs:** $N \geq 30$ independent timed runs per case.
- **Metrics:** mean, median, std dev, **95% CI**; show plots with error bars.
- **Fairness:** pin CPU governor (Linux performance; Windows set “High performance”).

- **Repeatability:** fixed PRNG seed **only** for synthetic data generation (never for keys/nonces).

Submission Checklist (attach to each lab ZIP)

- README.md with build & run steps for Windows & Linux
- CMakeLists.txt builds without editing paths or tasks.json
- Binaries for both OSes (or clear instructions to build)
- Source code, headers, tests, KAT files
- Report (.pdf/.docx) with tables/plots and analysis
- Self-grade rubric filled by the team
- (Optional) GUI project using your .dll/.so core

Appendix A — Academic Integrity & Ethics (paste into each report)

- Work is your own; cite libraries, KAT sources, and references.
- Attack demonstrations are limited to **your own test artifacts** in an **offline sandbox**.
- No testing against third-party data, networks, or services.
- Keys and secrets included in the repo are **test keys** only.
- You understand the difference between **vulnerability demonstration** and **malicious exploitation** and commit to the former strictly for learning.

Appendix B —Rubric Table (include & self-grade)

Criterion	Points	Self-score TA score
Correctness & KATs	35–40	
Security hygiene / misuse checks	10–20	
Cross-platform build & UX	10	
Performance study quality	10–20	

Criterion	Points	Self-score TA score
Report quality & clarity	5–10	
Negative tests	10–15	
Bonuses (if any)	+ up to 15	
Total	100 (+bonus)	